# Week-6 Assignment : Apache Spark

## Apache Spark Architecture, Data Processing & Performance Optimization using PySpark

**Dataset: S**ample Superstore Dataset

**Author:** Snehal A. Bhosale

**College:** Sanjivani College of Engineering, Kopargaon – 423603

**E-mail:** snehalbhosale1807@gmail.com

**Technology Used:**

Apache Spark (PySpark) Python Google Colab / Jupyter Notebook Apache Spark SQL CSV & Parquet File Formats

## Objective:

To understand Apache Spark Architecture and perform efficient data processing using DataFrames, transformations, filtering, schema handling, lazy evaluation, and optimized storage formats (CSV & Parquet). The assignment also focuses on Spark performance concepts such as DAG, Predicate Pushdown, Shuffle operations, and best practices for handling large datasets.

## Implementation:

## Step 1: Install Required Libraries

```
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/py
```

## Step 2: Import Required Libraries

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
```

## Step 3: Create Spark Session

```
spark = SparkSession.builder \
    .appName("Week6 Spark Assignment") \
    .getOrCreate()
```

Verify Spark

```
spark
```

**SparkSession - in-memory**

**SparkContext**

Spark UI

Version
v4.0.3
Master
local[*]
AppName
Week6 Spark Assignment

## Step 4: upload the CSV

```
from google.colab import files
uploaded = files.upload()
```

Browse... Sample - Superstore.csv

**Sample - Superstore.csv**(text/csv) - 2287806 bytes, last modified: n/a - 100% done

```
Saving Sample - Superstore.csv to Sample - Superstore.csv
```

## Step 5: Load the Dataset

```
df = spark.read.csv(
    "Sample - Superstore.csv",
    header=True,
    inferSchema=True
)
```

## Step 6: Explore the Dataset

```
df.show(5)
df.printSchema()
df.count()
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Cus
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darri
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
root
 |-- Row ID: integer (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Order Date: string (nullable = true)
 |-- Ship Date: string (nullable = true)
 |-- Ship Mode: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Customer Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Sales: string (nullable = true)
 |-- Quantity: string (nullable = true)
 |-- Discount: string (nullable = true)
 |-- Profit: double (nullable = true)

9994
```

## Step 7: Select and Filter Data

```
from pyspark.sql.functions import col

df_processed.select("Product ID", "Category", "Sales").show()

df_processed.filter(col("Category") == "Technology").show()

df_processed.filter((col("Sales") > 1000) & (col("Region") == "West")).show(

df_processed.filter((col("Region") == "South") | (col("Category") == "Furnit
```

```
+--------------+---------------+--------+
```

```
|     Product ID|       Category|   Sales|
+---------------+---------------+--------+
|FUR-BO-10001798|      Furniture|  261.96|
|FUR-CH-10000454|      Furniture|  731.94|
|OFF-LA-10000240|Office Supplies|   14.62|
|FUR-TA-10000577|      Furniture|957.5775|
|OFF-ST-10000760|Office Supplies|  22.368|
|FUR-FU-10001487|      Furniture|   48.86|
|OFF-AR-10002833|Office Supplies|    7.28|
|TEC-PH-10002275|     Technology| 907.152|
|OFF-BI-10003910|Office Supplies|  18.504|
|OFF-AP-10002892|Office Supplies|   114.9|
|FUR-TA-10001539|      Furniture|1706.184|
|TEC-PH-10002033|     Technology| 911.424|
|OFF-PA-10002365|Office Supplies|  15.552|
|OFF-BI-10003656|Office Supplies| 407.976|
|OFF-AP-10002311|Office Supplies|   68.81|
|OFF-BI-10000756|Office Supplies|   2.544|
|OFF-ST-10004186|Office Supplies|  665.88|
|OFF-ST-10000107|Office Supplies|    55.5|
|OFF-AR-10003056|Office Supplies|    8.56|
|TEC-PH-10001949|     Technology|  213.48|
+---------------+---------------+--------+
only showing top 20 rows
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    36|CA-2016-117590| 12/8/2016|12/10/2016|   First Class|   GH-14485|
|    41|CA-2015-117415|12/27/2015|12/31/2015|Standard Class|   SN-20710|
|    42|CA-2017-120999| 9/10/2017| 9/15/2017|Standard Class|   LC-16930|    L
|    45|CA-2016-118255| 3/11/2016| 3/13/2016|   First Class|   ON-18715|
|    48|CA-2016-169194| 6/20/2016| 6/25/2016|Standard Class|   LH-16900|    L
|    49|CA-2016-169194| 6/20/2016| 6/25/2016|Standard Class|   LH-16900|    L
|    55|CA-2016-105816|12/11/2016|12/17/2016|Standard Class|   JM-15265|    J
|    60|CA-2016-111682| 6/17/2016| 6/18/2016|   First Class|   TB-21055|   Te
|    63|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
|    69|CA-2014-106376| 12/5/2014|12/10/2014|Standard Class|   BS-11590|
|    87|CA-2017-155558|10/26/2017| 11/2/2017|Standard Class|   PG-18895|
|    91|CA-2016-109806| 9/17/2016| 9/22/2016|Standard Class|   JS-15685|
|   101|CA-2016-158568| 8/29/2016|  9/2/2016|Standard Class|   RB-19465|
|   104|US-2015-156867|11/13/2015|11/17/2015|Standard Class|   LC-16870|
|   107|CA-2017-119004|11/23/2017|11/28/2017|Standard Class|   JM-15250|
|   108|CA-2017-119004|11/23/2017|11/28/2017|Standard Class|   JM-15250|
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|    C
+------+--------------+----------+----------+--------------+-----------+-----
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|  Bro
|    25|CA-2015-106320| 9/25/2015| 9/30/2015|Standard Class|   EB-13870|
|    68|CA-2014-106376| 12/5/2014|12/10/2014|Standard Class|   BS-11590|    B
```

```
|   252|CA-2016-145625| 9/11/2016| 9/17/2016|Standard Class|   KC-16540|  Kel
|   283|CA-2015-130890| 11/2/2015| 11/6/2015|Standard Class|   JO-15280|    J
```

## Step 8: Modify the DataFrame

```
from pyspark.sql.functions import col, when
from pyspark.sql.types import DoubleType

# Helper function to safely cast to DoubleType, converting non-numeric strin
def safe_cast_to_double(column_name):
    # Ensure raw string for regex to avoid SyntaxWarning
    return when(col(column_name).cast("string").rlike(r"^-?\d*\.?\d+$"),
                col(column_name).cast(DoubleType())).otherwise(None)

# Rename Sales to Total_Sales
df = df.withColumnRenamed("Sales", "Total_Sales")

# Apply safe casting to Total_Sales, Quantity, and Discount
# This ensures that any malformed strings in these columns are converted to
df = df.withColumn("Total_Sales", safe_cast_to_double("Total_Sales"))
df = df.withColumn("Quantity", safe_cast_to_double("Quantity"))
df = df.withColumn("Discount", safe_cast_to_double("Discount"))

# Calculate Final_Price now that Total_Sales is correctly typed
df = df.withColumn("Final_Price", col("Total_Sales") * 1.18)

df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Cus
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darri
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
```

## Step 9: Handle Missing Values

```
from pyspark.sql.functions import col, when, count

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

```
df = df.na.drop()
```

```
+------+--------+----------+---------+---------+-----------+-------------+---
|Row ID|Order ID|Order Date|Ship Date|Ship Mode|Customer ID|Customer Name|Seg
+------+--------+----------+---------+---------+-----------+-------------+---
|     0|       0|         0|        0|        0|          0|            0|
+------+--------+----------+---------+---------+-----------+-------------+---
```

## Step 10: Perform Transformations and Actions

```
# Transformations
filtered_df = df.filter(col("Region") == "West").select("Customer Name", "To

# Actions
filtered_df.show()
filtered_df.count()
```

```
+------------------+-----------+
|     Customer Name|Total_Sales|
+------------------+-----------+
|   Darrin Van Huff|      14.62|
|   Brosina Hoffman|      48.86|
|   Brosina Hoffman|       7.28|
|   Brosina Hoffman|    907.152|
|   Brosina Hoffman|     18.504|
|   Brosina Hoffman|      114.9|
|   Brosina Hoffman|   1706.184|
|   Brosina Hoffman|    911.424|
|      Irene Maddox|    407.976|
|   Alejandro Grove|       55.5|
|Zuschuss Donatelli|       8.56|
|Zuschuss Donatelli|     213.48|
|Zuschuss Donatelli|      22.72|
|       Emily Burns|    1044.63|
|     Eric Hoffmann|     11.648|
|     Eric Hoffmann|      90.57|
|      Ruben Ausman|      77.88|
|      Kunst Miller|      13.98|
|      Kunst Miller|     25.824|
|      Kunst Miller|     146.73|
+------------------+-----------+
only showing top 20 rows
3203
```

## Step 11: Save Data in CSV and Parquet

```
df.write.mode("overwrite").option("header", True).csv("output/csv_output")

df.write.mode("overwrite").parquet("output/parquet_output")
```

## Step 12: Read Parquet and Demonstrate Predicate Pushdown

```
parquet_df = spark.read.parquet("output/parquet_output")

parquet_df.filter(col("Region") == "West").show()

parquet_df.filter(col("Region") == "West").explain(True)
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|   Da
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    14|CA-2016-161389| 12/5/2016|12/10/2016|Standard Class|   IM-15070|
|    18|CA-2014-167164| 5/13/2014| 5/15/2014|  Second Class|   AG-10270|   Al
|    19|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    21|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    25|CA-2015-106320| 9/25/2015| 9/30/2015|Standard Class|   EB-13870|
|    26|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    43|CA-2016-101343| 7/17/2016| 7/22/2016|Standard Class|   RA-19885|
|    63|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
|    64|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
|    65|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
== Parsed Logical Plan ==
'Filter '`=`('Region, West)
+- Relation [Row ID#3744,Order ID#3745,Order Date#3746,Ship Date#3747,Ship Mo

== Analyzed Logical Plan ==
Row ID: int, Order ID: string, Order Date: string, Ship Date: string, Ship Mo
Filter (Region#3756 = West)
+- Relation [Row ID#3744,Order ID#3745,Order Date#3746,Ship Date#3747,Ship Mo

== Optimized Logical Plan ==
Filter (isnotnull(Region#3756) AND (Region#3756 = West))
+- Relation [Row ID#3744,Order ID#3745,Order Date#3746,Ship Date#3747,Ship Mo

== Physical Plan ==
*(1) Filter (isnotnull(Region#3756) AND (Region#3756 = West))
+- *(1) ColumnarToRow
   +- FileScan parquet [Row ID#3744,Order ID#3745,Order Date#3746,Ship Date#3
```

## Step 13: Compare CSV and Parquet Performance

```
# Read CSV
csv_df = spark.read.csv(
    "/content/Sample - Superstore.csv",
    header=True,
    inferSchema=True
)

# Read Parquet
parquet_df = spark.read.parquet("output/parquet_output")

# Compare number of records
print("CSV Records:", csv_df.count())
print("Parquet Records:", parquet_df.count())

# Display sample data
csv_df.show(5)
parquet_df.show(5)
```

```
CSV Records: 9994
Parquet Records: 9994
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Cus
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darri
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Cus
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darri
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
```

## Step 14: Build the Spark Data Pipeline

```
from pyspark.sql.functions import col

pipeline df = (
```

```
    df
    .filter(col("Category") == "Technology")
    .withColumn("Final_Price", col("Sales") * 1.18)
)

pipeline_df.write.mode("overwrite").parquet("output/final_pipeline")
```

```
pipeline_df.printSchema()
```

```
root
 |-- Row ID: integer (nullable = true)
 |-- Order ID: string (nullable = true)
 |-- Order Date: string (nullable = true)
 |-- Ship Date: string (nullable = true)
 |-- Ship Mode: string (nullable = true)
 |-- Customer ID: string (nullable = true)
 |-- Customer Name: string (nullable = true)
 |-- Segment: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- State: string (nullable = true)
 |-- Postal Code: integer (nullable = true)
 |-- Region: string (nullable = true)
 |-- Product ID: string (nullable = true)
 |-- Category: string (nullable = true)
 |-- Sub-Category: string (nullable = true)
 |-- Product Name: string (nullable = true)
 |-- Sales: double (nullable = true)
 |-- Quantity: integer (nullable = true)
 |-- Discount: double (nullable = true)
 |-- Profit: double (nullable = true)
 |-- Final_Price: double (nullable = true)
```

Verify the saved data

```
spark.read.parquet("output/final_pipeline").show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    36|CA-2016-117590| 12/8/2016|12/10/2016|   First Class|   GH-14485|
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
```

## Step 15: Performance Insights & Best Practices

**Performance Insights**

1. Spark uses Lazy Evaluation, executing transformations only when an action (show(), count(), etc.) is called.
2. Parquet files provide faster query performance and consume less storage than CSV due to their columnar format and compression.
3. Predicate Pushdown filters data at the storage level, reducing the amount of data loaded into memory.
4. Wide Transformations (e.g., groupBy()) involve data shuffling, while Narrow Transformations (e.g., filter(), select()) do not.

**Best Practices**

1. Use .show(5) for previewing data instead of .collect() on large datasets.
2. Define the schema or use inferSchema=True while reading data.
3. Prefer Parquet over CSV for storing processed datasets.
4. Filter data as early as possible to minimize processing.
5. Avoid unnecessary transformations and repeated actions to improve performance.

# Week-6 : Spark Assignment Questions and Answers (Q1 to Q15)

## Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

## Answer:

Spark follows a **Master-Worker Architecture** where different components work together to execute distributed applications.

| Component | Role |
|---|---|
| **Driver** | The main program that creates the Spark Session, converts code into tasks, schedules jobs, an |
| **Cluster Manager** | Allocates resources (CPU and memory) across the cluster and launches Executors. Examples i |
| **Executor** | Worker processes running on cluster nodes. They execute tasks, store cached data, and send |

### Execution Flow

```
Driver Program
      |
      ▼
Cluster Manager
      |
      ▼
Executors
      |
      ▼
Process Data → Return Results
```

---

## Q2. How does Spark's Lazy Evaluation strategy improve performance?

## Answer:

Spark does **not execute transformations immediately**. Instead, it records them and creates a **Directed Acyclic Graph (DAG)**. Execution begins only when an **Action** (such as `show()` or `count()`) is called.

### Advantages

- Minimizes unnecessary computations.
- Optimizes execution using DAG.
- Reduces disk I/O.
- Combines multiple transformations into a single execution plan.

### Example

```
filtered_df = df.filter(col("Region") == "West") \
                .select("Customer Name", "Sales")

# No execution yet

filtered_df.show()      # Action triggers execution
```

```
+------------------+--------+
|     Customer Name|   Sales|
```

```
+------------------+--------+
|   Darrin Van Huff|   14.62|
|   Brosina Hoffman|   48.86|
|   Brosina Hoffman|    7.28|
|   Brosina Hoffman| 907.152|
|   Brosina Hoffman|  18.504|
|   Brosina Hoffman|   114.9|
|   Brosina Hoffman|1706.184|
|   Brosina Hoffman| 911.424|
|      Irene Maddox| 407.976|
|   Alejandro Grove|    55.5|
|Zuschuss Donatelli|    8.56|
|Zuschuss Donatelli|  213.48|
|Zuschuss Donatelli|   22.72|
|       Emily Burns| 1044.63|
|     Eric Hoffmann|  11.648|
|     Eric Hoffmann|   90.57|
|      Ruben Ausman|   77.88|
|      Kunst Miller|   13.98|
|      Kunst Miller|  25.824|
|      Kunst Miller|  146.73|
+------------------+--------+
only showing top 20 rows
```

# Q3. Read a CSV file with Header and Infer Schema

## Answer

- `header=True` → Uses the first row as column names.
- `inferSchema=True` → Automatically detects column data types.

---

```
df = spark.read.csv(
    "/content/Sample - Superstore.csv",
    header=True,
    inferSchema=True
)
```

# Q4. Difference between CSV and Parquet

## Answer

| Feature | CSV | Parquet |
|---|---|---|
| Storage Type | Row-based | Columnar |

| | | |
|---|---|---|
| Compression | No | Yes |
| Schema | Not Stored | Stored |
| Query Performance | Slower | Faster |
| Storage Size | Larger | Smaller |
| Analytics | Less Efficient | Highly Efficient |

### Why Parquet is Faster?

Parquet reads only the required columns, reducing disk I/O and improving query performance. It also supports **Predicate Pushdown**, making it ideal for big data analytics.

---

## Q5. Select `product_id` and `price` where category is 'Electronics'

### Answer

```
from pyspark.sql.functions import col

df.filter(col("Category") == "Technology") \
  .select("Product ID", "Sales") \
  .show()
```

```
+---------------+--------+
|     Product ID|   Sales|
+---------------+--------+
|TEC-PH-10002275| 907.152|
|TEC-PH-10002033| 911.424|
|TEC-PH-10001949|  213.48|
|TEC-AC-10003027|   90.57|
|TEC-PH-10004977|1097.544|
|TEC-PH-10000486| 371.168|
|TEC-PH-10004093| 147.168|
|TEC-AC-10000171|   45.98|
|TEC-AC-10002167|      45|
|TEC-PH-10003988|    21.8|
|TEC-PH-10002447| 1029.95|
|TEC-AC-10002167|      30|
|TEC-AC-10004633|   13.98|
|TEC-PH-10002726| 167.968|
|TEC-AC-10001998|   19.99|
|TEC-PH-10004093|  73.584|
|TEC-AC-10001767|  95.976|
|TEC-AC-10001552| 238.896|
|TEC-AC-10003499|  74.112|
|TEC-PH-10002844|  27.992|
```

```
+---------------+--------+
only showing top 20 rows
```

# Q6. Rename a column and cast data type

## Answer

```
from pyspark.sql.functions import col

# Rename Sales to Total_Sales
df = df.withColumnRenamed("Sales", "Total_Sales")

# Cast Total_Sales to Double
df = df.withColumn(
    "Total_Sales",
    col("Total_Sales").cast("double")
)

df.show(5)
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|  Cus
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|    C
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|Darri
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335| Sean
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 5 rows
```

# Q7. How does Spark's DAG provide Fault Tolerance?

## Answer

Spark maintains a **Lineage Graph (DAG)** that records every transformation applied to the data.

If an Executor or worker node fails:

- Spark identifies the lost partition.
- Recomputes only the missing partition using the lineage graph.
- Does **not** recompute the entire dataset

Does **not** recompute the entire dataset.

### Benefits

- Automatic recovery.
- No data replication required.
- Efficient fault recovery.

---

# Q8. Filter orders where Status is 'Completed' and Amount > 1000

### Answer

```
from pyspark.sql.functions import col, when
from pyspark.sql.types import DoubleType

# Helper function to safely cast to DoubleType, converting non-numeric strin
def safe_cast_to_double(column_name):
    # Ensure raw string for regex to avoid SyntaxWarning
    return when(col(column_name).cast("string").rlike(r"^-?\d*\.?\d+$"),
                col(column_name).cast(DoubleType())).otherwise(None)

# Apply safe casting to the 'Sales' column before filtering
df_fresh_processed = df_fresh.withColumn("Sales", safe_cast_to_double("Sales

df_fresh_processed.filter(col("Sales") > 1000).show()
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|    C
+------+--------------+----------+----------+--------------+-----------+-----
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|  Bro
|    25|CA-2015-106320| 9/25/2015| 9/30/2015|Standard Class|   EB-13870|
|    28|US-2015-150630| 9/17/2015| 9/21/2015|Standard Class|   TB-21520|  Tra
|    36|CA-2016-117590| 12/8/2016|12/10/2016|   First Class|   GH-14485|
|    55|CA-2016-105816|12/11/2016|12/17/2016|Standard Class|   JM-15265|   Ja
|    68|CA-2014-106376| 12/5/2014|12/10/2014|Standard Class|   BS-11590|    B
|   150|CA-2016-114489| 12/5/2016| 12/9/2016|Standard Class|   JE-16165|   Ju
|   166|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-11140|
|   168|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-11140|
|   216|CA-2015-146262|  1/2/2015|  1/9/2015|Standard Class|   VW-21775|  Vic
|   245|CA-2014-131926|  6/1/2014|  6/6/2014|  Second Class|   DW-13480|    D
|   248|CA-2014-131926|  6/1/2014|  6/6/2014|  Second Class|   DW-13480|    D
|   252|CA-2016-145625| 9/11/2016| 9/17/2016|Standard Class|   KC-16540|  Kel
|   263|US-2014-106992| 9/19/2014| 9/21/2014|  Second Class|   SB-20290|
|   264|US-2014-106992| 9/19/2014| 9/21/2014|  Second Class|   SB-20290|
|   282|US-2015-161991| 9/26/2015| 9/28/2015|  Second Class|   SC-20725|Steve
|   283|CA-2015-130890| 11/2/2015| 11/6/2015|Standard Class|   JO-15280|    J
```

```
|   319|CA-2014-164973| 11/4/2014| 11/9/2014|Standard Class|   NM-18445|
|   354|CA-2016-129714|  9/1/2016|  9/3/2016|   First Class|   AB-10060|  Ada
|   370|CA-2016-155516|10/21/2016|10/21/2016|      Same Day|   MK-17905|  Mic
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
```

# Q9. Explain Predicate Pushdown

## Answer

**Predicate Pushdown** is an optimization technique used mainly with **Parquet** and other columnar storage formats.

Instead of loading the entire dataset into memory, Spark sends the filter condition directly to the storage layer. Only matching rows are read.

## Benefits

- Reads less data.
- Reduces memory usage.
- Improves query execution speed.
- Minimizes disk I/O.

## Example

```
parquet_df.filter(col("Region") == "West").show()
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|   Da
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    14|CA-2016-161389| 12/5/2016|12/10/2016|Standard Class|   IM-15070|
|    18|CA-2014-167164| 5/13/2014| 5/15/2014|  Second Class|   AG-10270|   Al
|    19|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    21|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    25|CA-2015-106320| 9/25/2015| 9/30/2015|Standard Class|   EB-13870|
|    26|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    27|CA-2016-121755| 1/16/2016| 1/20/2016|  Second Class|   EH-13945|
|    43|CA-2016-101343| 7/17/2016| 7/22/2016|Standard Class|   RA-19885|
```

```
|    63|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
|    64|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
|    65|CA-2015-135545|11/24/2015|11/30/2015|Standard Class|   KM-16720|
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
```

# Q10. Add a new column `final_price`

## Answer

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("Total_Sales") * 1.18
)

df.show()
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|   Da
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335|    S
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335|    S
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    13|CA-2017-114412| 4/15/2017| 4/20/2017|Standard Class|   AA-10480|
|    14|CA-2016-161389| 12/5/2016|12/10/2016|Standard Class|   IM-15070|
|    15|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-14815|
|    16|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-14815|
|    17|CA-2014-105893|11/11/2014|11/18/2014|Standard Class|   PK-19075|
|    18|CA-2014-167164| 5/13/2014| 5/15/2014|  Second Class|   AG-10270|   Al
|    19|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
```

# Q11. Difference between Transformations and Actions

## Answer

| Transformations | Actions |
|---|---|
| Create a new DataFrame | Execute the computation |
| Lazy Evaluation | Trigger execution |
| Return a DataFrame | Return values or display results |

## Examples

```
#### Transformations

df.filter(col("Total_Sales") > 1000)

df.select("Customer Name", "Total_Sales")


#### Actions

df.show()

df.count()
```

```
+------+--------------+----------+----------+--------------+-----------+-----
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|
+------+--------------+----------+----------+--------------+-----------+-----
|     1|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|
|     2|CA-2016-152156| 11/8/2016|11/11/2016|  Second Class|   CG-12520|
|     3|CA-2016-138688| 6/12/2016| 6/16/2016|  Second Class|   DV-13045|   Da
|     4|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335|    S
|     5|US-2015-108966|10/11/2015|10/18/2015|Standard Class|   SO-20335|    S
|     6|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     7|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     8|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|     9|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    10|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    11|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    12|CA-2014-115812|  6/9/2014| 6/14/2014|Standard Class|   BH-11710|   Br
|    13|CA-2017-114412| 4/15/2017| 4/20/2017|Standard Class|   AA-10480|
|    14|CA-2016-161389| 12/5/2016|12/10/2016|Standard Class|   IM-15070|
|    15|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-14815|
|    16|US-2015-118983|11/22/2015|11/26/2015|Standard Class|   HP-14815|
|    17|CA-2014-105893|11/11/2014|11/18/2014|Standard Class|   PK-19075|
|    18|CA-2014-167164| 5/13/2014| 5/15/2014|  Second Class|   AG-10270|   Al
|    19|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
|    20|CA-2014-143336| 8/27/2014|  9/1/2014|  Second Class|   ZD-21925|Zusch
+------+--------------+----------+----------+--------------+-----------+-----
only showing top 20 rows
9994
```

# Q12. Load Parquet → Filter Null Values → Save as CSV

## Answer

```
from pyspark.sql.functions import col

parquet_df_read = spark.read.parquet("output/parquet_output")

parquet_df_read.filter(
    col("Sales").isNotNull()
).write.mode("overwrite") \
 .option("header", True) \
 .csv("output/filtered_parquet_to_csv")
```

# Q13. Difference between Client Mode and Cluster Mode

## Answer

| Client Mode | Cluster Mode |
|---|---|
| Driver runs on the local machine. | Driver runs inside the cluster. |
| Suitable for development and testing. | Suitable for production environments. |
| If the client disconnects, the application stops. | Continues running even if the client disconnects. |
| Easier debugging. | Better scalability and fault tolerance. |

# Q14. Filter Region is 'North' OR Priority is 'High'

## Answer

*Note:The Sample Superstore dataset does not contain a Priority column. Therefore, the query has been adapted to filter records based on the available Region column.*

```
df.filter(
    col("Region") == "North"
).show()
```

```
+------+--------+----------+---------+---------+-----------+-------------+---
|Row ID|Order ID|Order Date|Ship Date|Ship Mode|Customer ID|Customer Name|Seg
+------+--------+----------+---------+---------+-----------+-------------+---
+------+--------+----------+---------+---------+-----------+-------------+---
```

# Q15. Why use `.show(5)` instead of `.collect()` on large datasets?

## Answer

The `.show(5)` function displays only the first **five rows** of a DataFrame, whereas `.collect()` retrieves **all rows** from every Executor to the Driver.

For very large datasets (GBs or TBs), using `.collect()` can:

- Consume excessive Driver memory.
- Cause **OutOfMemoryError**.
- Slow down application performance.
- Increase network traffic.

# Key Learning Outcomes

- Understood the **Apache Spark Architecture**, including the roles of the Driver, Cluster Manager, and Executors.
- Learned Spark's **Lazy Evaluation** and **DAG execution** for optimized processing.
- Performed DataFrame operations such as **reading, filtering, selecting, renaming, casting, and adding new columns**.
- Applied **data cleaning** techniques by handling null values and schema management.
- Differentiated between **Transformations** and **Actions** in Spark.
- Compared **CSV** and **Parquet** file formats and understood the performance benefits of Parquet.
- Explored **Predicate Pushdown**, **Shuffle**, and other Spark performance optimization concepts.
- Built an end-to-end **Spark data pipeline** for reading, transforming, filtering, and saving processed data.
- Followed Spark best practices for efficiently processing large datasets.

- Followed Spark best practices for efficiently processing large datasets.

## Conclusion

This assignment provided practical experience with **Apache Spark Fundamentals** and **PySpark DataFrame operations**. It covered Spark architecture, data transformations, schema handling, performance optimization, and file formats. By implementing an end-to-end data pipeline, the assignment strengthened the understanding of scalable data processing and industry-standard Spark practices, forming a strong foundation for advanced data engineering and big data applications.